

Emotion Detection & Learning Support Engine

Project Description:

The Emotion Detection & Learning Support Engine is an AI-powered platform that transforms a learner's free-text description of a study challenge into an emotion-aware, supportive response. The platform ingests raw student input, predicts nuanced emotional states (Bored, Confident, Confused, Curious, Frustrated) using deep learning classifiers, and delivers tailored tips, next steps, and encouragement.

Built using **BiLSTM**, **BERT**, **Streamlit**, and the **Gemini AI** API, the application combines deep learning emotion classification with rule-based keyword enhancement and generative guidance. The system also logs interactions to CSV for continuous learning, visualizes emotion trends through an analytics dashboard, and supports side-by-side comparison between the BiLSTM and BERT models.

Scenarios

Scenario 1:

An instructor or teaching assistant pastes a student's message into the platform. The system automatically classifies the underlying emotion — for example, Confused or Frustrated — and generates a supportive, field-specific reply. Dual-model comparison and mixed-emotion detection help the TA triage requests faster and respond with consistent empathy.

Scenario 2:

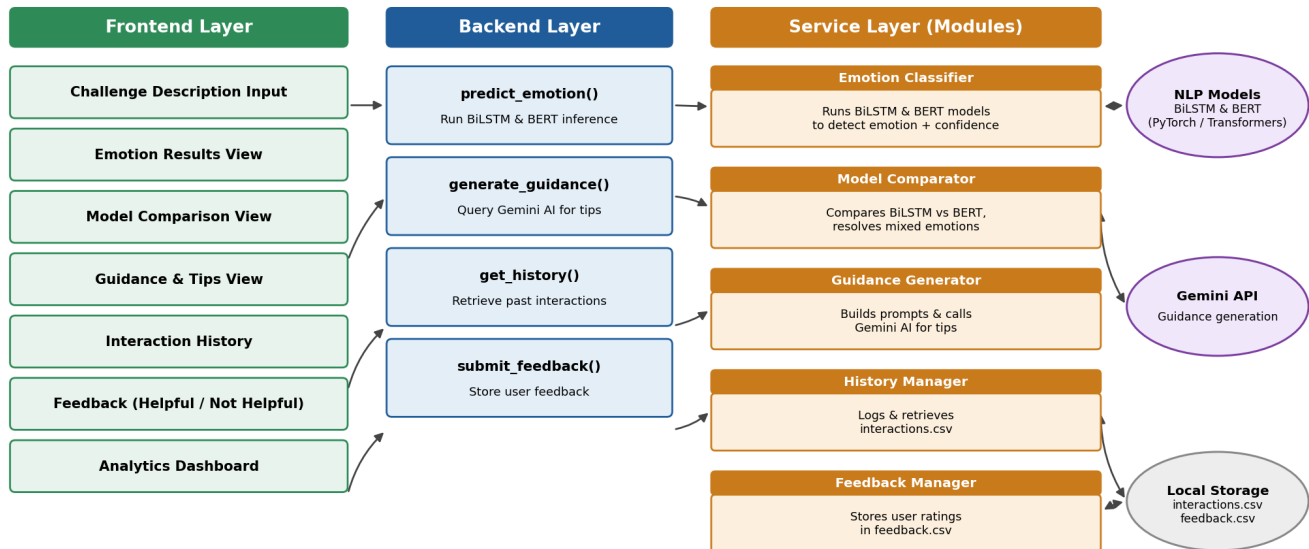
A student working independently types "I'm lost on recursion." The system flags Confused, surfaces clarity-focused tips, and provides an encouraging next step. Mixed-emotion output (e.g., Curious + Confused) helps the learner self-reflect, while saved interactions build a history for progress tracking.

Scenario 3:

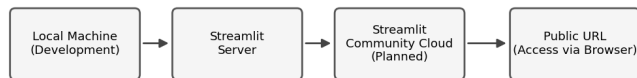
A corporate trainer reviews a batch of learner check-ins after a workshop. The analytics dashboard visualizes emotion trends across the cohort, helping the trainer identify which topics are causing widespread frustration or confusion and adjust the next session accordingly.

Technical Architecture

Technical Architecture: Emotion Detection & Learning Support Engine



Deployment



Key Technologies

- Frontend: Streamlit
- Emotion Models: BiLSTM (PyTorch), BERT (Hugging Face Transformers)
- Guidance Generation: Gemini AI
- Storage: Local CSV Files (interactions, feedback)
- Deployment: Streamlit Community Cloud (planned)

Description:

The Emotion Detection & Learning Support Engine follows a modular three-tier architecture.

- The **Frontend Layer** consists of a Streamlit-based user interface.
- The **Backend Layer** handles inference requests, guidance generation, and logging.
- The **Service Layer** contains reusable modules for emotion classification, model comparison, guidance generation, history logging, and feedback management.

The architecture enables scalability, maintainability, and modular development.

Pre-requisites

- Python Programming Knowledge
 - PyTorch Framework Knowledge
 - Hugging Face Transformers Library
 - Streamlit Framework Knowledge
 - Natural Language Processing & Deep Learning Concepts
 - Generative AI / Prompt Engineering (Gemini API)
 - Git and GitHub Basics
 - Virtual Environment Management
-

Project Workflow

Milestone 1: Environment Setup and Architecture Design

Activity 1.1: Define Project Requirements

- Identify learner requirements.
- Define emotion-support use cases.
- Finalize project scope.

Activity 1.2: Design System Architecture

- Create architectural diagrams.
- Define interactions between frontend, backend, and services.

Activity 1.3: Set Up Development Environment

- Install Python.
- Create virtual environment.
- Install dependencies (PyTorch, Transformers, Streamlit, google-generativeai).

Activity 1.4: Create Project Structure

- Create folders for:
 - o data/
 - o models/
 - o src/
 - o app/
 - o logs/
 - o tests/

Milestone 2: Model & Backend Development

Activity 2.1: Prepare Emotion Dataset

- Collect and label student text samples across the five emotion categories.
 - Clean and split data for training and evaluation.
-

Activity 2.2: Develop BiLSTM Emotion Classifier

Functions:

```
train_bilstm()  
predict_bilstm()
```

Responsibilities:

- Train the BiLSTM model on labeled emotion data.
 - Predict emotion probabilities from learner text.
-

Activity 2.3: Develop BERT Emotion Classifier

Functions:

```
fine_tune_bert()  
predict_bert()
```

Responsibilities:

- Fine-tune a BERT transformer on the same dataset.
 - Predict emotion probabilities for comparison against BiLSTM.
-

Activity 2.4: Develop Guidance Generation Module

Integrate Gemini AI.

Functions:

```
generate_guidance()
```

Responsibilities:

- Send the detected emotion and context to Gemini AI.
 - Retrieve personalized tips, next steps, and encouragement.
-

Milestone 3: Frontend Development

Activity 3.1: Develop Streamlit Interface

Create user-friendly pages for:

- Challenge input.
 - Emotion results display.
 - Model comparison view.
 - Guidance & tips display.
-

Activity 3.2: Add User Interaction Features

Implement:

- AI response toggle.
- Feedback (helpful / not helpful).
- Interaction history.
- Loading animations.

Milestone 4: Data Storage and Logging

Activity 4.1: Interaction History

Store generated interactions in:

`interactions.csv`

Responsibilities:

- Save challenge text, predictions, and guidance.
 - Retrieve historical interactions.
-

Activity 4.2: Feedback Collection

Store user ratings in:

`feedback.csv`

Responsibilities:

- Save helpful / not helpful ratings.
 - Analyze feedback trends for the dashboard.
-

Milestone 5: Testing and Validation

Activity 5.1: Unit Testing

Create test files:

`test_bilstm_model.py`

`test_bert_model.py`

`test_guidance_generator.py`

Use:

`pytest -v`

Validate:

- Emotion prediction accuracy.
 - Model comparison logic.
 - Guidance generation.
 - Logging functions.
-

Activity 5.2: Manual & Dashboard Testing

Access the Streamlit app locally.

Test:

- Mixed-emotion detection.
 - Analytics dashboard rendering.
 - Feedback submission flow.
-

Milestone 6: Deployment and Documentation

Activity 6.1: Local Deployment

Run Streamlit:

`streamlit run app.py`

Activity 6.2: Version Control

Upload project to GitHub.

Repository includes:

- Source code.
- Documentation.
- Trained model files or download scripts.
- Tests.
- README.

Exploring the Application Pages

Home Page

Description:

The homepage allows users to:

- Enter their study challenge description.
 - Submit the text for emotion analysis.
 - View the generated response.
-

Emotion Results Section

Description:

Displays the predicted emotion(s) and confidence scores.

Example:

Confused (62%)
Curious (28%)
Frustrated (10%)

Model Comparison Section

Description:

Displays BiLSTM vs BERT predictions side-by-side.

Example:

BiLSTM: Confused
BERT: Confused + Curious

Guidance & Tips Section

Description:

Displays Gemini AI-generated personalized tips and next steps.

Example:

"Try breaking the problem into smaller steps."
"Revisit the base case before tackling recursion."

History Section

Description:

Displays previously logged interactions and predictions.

Analytics Dashboard

Description:

Visualizes emotion trends over time.

Conclusion

The Emotion Detection & Learning Support Engine successfully demonstrates how deep learning and generative AI can improve learner support. The system assists students and educators by detecting nuanced emotional states, generating personalized guidance, comparing model predictions, and maintaining interaction history.

The modular architecture built using BiLSTM, BERT, and Streamlit ensures maintainability and scalability. Future enhancements include cloud deployment, user authentication, database integration, multilingual support, and further refinement of the Gemini AI guidance prompts.

This project highlights the practical application of AI in enhancing empathetic communication and personalized academic support.